

Kafka production issues

06/05/2026 08:24:13

<https://claude.ai/chat/a2432a7e-b16b-4582-a60e-9f707a4417aa>

Apache kafka in java production issues or what are the issues we may expect...

Apache Kafka in Java – Production Issues & Interview Q&A --- ## PART 1:...



Apache kafka in java production issues or what are the issues we may expect and how we can resolve it
And what are the kafka issues we may face (at least 50 q&a)

6/2/2026, 5:46:45 PM



Apache Kafka in Java – Production Issues & Interview Q&A

PART 1: COMMON PRODUCTION ISSUES & RESOLUTIONS

1. Consumer Lag / Slow Consumption

Issue: Consumers fall behind producers; lag grows unboundedly.

Causes:

- Heavy processing logic inside `poll()` loop
- Under-partitioned topics
- Single-threaded consumers

Resolution:

```
// Increase max.poll.records to batch process
props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, 500);

// Offload processing to thread pool, commit after processing
ExecutorService executor = Executors.newFixedThreadPool(10);

while (true) {
    ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(100));
    for (ConsumerRecord<String, String> record : records) {
```

```
        executor.submit(() -> processRecord(record));
    }
    consumer.commitAsync();
}
```

2. Consumer Rebalancing Storms

Issue: Frequent rebalances causing all consumers to stop and rejoin the group, creating processing gaps.

Causes:

- `max.poll.interval.ms` too short for slow processing
- Network instability causing missed heartbeats

Resolution:

```
// Increase poll interval and heartbeat tuning
props.put(ConsumerConfig.MAX_POLL_INTERVAL_MS_CONFIG, 600000); //
10 min
props.put(ConsumerConfig.SESSION_TIMEOUT_MS_CONFIG, 30000);
props.put(ConsumerConfig.HEARTBEAT_INTERVAL_MS_CONFIG, 10000);

// Use Static Membership to avoid rebalance on restart
props.put(ConsumerConfig.GROUP_INSTANCE_ID_CONFIG, "consumer-
instance-1");
```

3. Message Loss (Producer Side)

Issue: Messages are sent but never received by consumers.

Causes:

- `acks=0` or `acks=1` with broker failure
- No retry configured
- Fire-and-forget pattern

Resolution:

```
// Guaranteed delivery config
props.put(ProducerConfig.ACKS_CONFIG, "all");           // Wait for
all ISR replicas
props.put(ProducerConfig.RETRIES_CONFIG, 3);
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true); //
Exactly-once semantics
props.put(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION, 1);

// Always use callback to detect failures
producer.send(record, (metadata, exception) -> {
    if (exception != null) {
        log.error("Send failed for record: {}", record.key(),
exception);
        // Push to dead-letter topic or retry queue
    }
});
```

4. Duplicate Messages (At-Least-Once)

Issue: Same message processed more than once due to consumer crash before offset commit.

Resolution:

```
// Option 1: Idempotent consumer using DB unique key
String messageId = record.key() + "-" + record.offset();
if (!processedIds.contains(messageId)) {
    processRecord(record);
    processedIds.add(messageId);
}

// Option 2: Exactly-once with transactions
props.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "claims-producer-
1");
```

```
producer.initTransactions();

producer.beginTransaction();
try {
    producer.send(new ProducerRecord<>("claims-output", key,
value));
    producer.commitTransaction();
} catch (Exception e) {
    producer.abortTransaction();
}
```

5. Offset Commit Issues

Issue: Offsets committed before processing completes → data loss on crash.

Resolution:

```
// Disable auto-commit; commit only after successful processing
props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);

for (ConsumerRecord<String, String> record : records) {
    try {
        processRecord(record);
        // Commit specific offset after processing
        Map<TopicPartition, OffsetAndMetadata> offsetsToCommit = new
HashMap<>();
        offsetsToCommit.put(
            new TopicPartition(record.topic(), record.partition()),
            new OffsetAndMetadata(record.offset() + 1)
        );
        consumer.commitSync(offsetsToCommit);
    } catch (Exception e) {
        log.error("Processing failed, skipping commit", e);
    }
}
```

6. Serialization / Deserialization Errors

Issue: `SerializationException` or poison pill messages that crash consumers.

Resolution:

```
// Use ErrorHandlingDeserializer in Spring Kafka
@Bean
public ConsumerFactory<String, ClaimEvent> consumerFactory() {
    Map<String, Object> props = new HashMap<>();
    props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
bootstrapServers);

    ErrorHandlingDeserializer<ClaimEvent> valueDeserializer =
        new ErrorHandlingDeserializer<>(new JsonSerializer<>
(ClaimEvent.class));

    return new DefaultKafkaConsumerFactory<>(props,
        new StringDeserializer(), valueDeserializer);
}

// Dead letter topic for poison pills
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<String,
Object> template) {
    DeadLetterPublishingRecoverer recoverer =
        new DeadLetterPublishingRecoverer(template);
    return new DefaultErrorHandler(recoverer, new
FixedBackOff(1000L, 3));
}
```

7. Broker Leader Election Delays

Issue: During broker restart, leader election takes too long; producers get

`NotLeaderForPartitionException`.

Resolution:

- Set `unclean.leader.election.enable=false` to prevent data loss
 - Tune `replica.lag.time.max.ms` (default 30s) — reduce if replicas fall behind fast
 - Producer-side: set `retries=5` and `retry.backoff.ms=500`
-

8. Log Retention & Disk Full

Issue: Broker disk fills up; topic data loss or broker crash.

Resolution:

```
# Per-topic retention config
log.retention.hours=168          # 7 days
log.retention.bytes=10737418240 # 10 GB per partition
log.segment.bytes=1073741824    # Roll segments at 1 GB
```

```
// Programmatically update via AdminClient
AdminClient adminClient = AdminClient.create(props);
ConfigEntry retention = new ConfigEntry("retention.ms",
    "604800000");
Map<ConfigResource, Config> configs = new HashMap<>();
configs.put(
    new ConfigResource(ConfigResource.Type.TOPIC, "claims-events"),
    new Config(Collections.singleton(retention))
);
adminClient.alterConfigs(configs);
```

9. Spring Kafka Listener Container Issues

Issue: `@KafkaListener` stops consuming silently after exceptions.

Resolution:

```
@Bean
public ConcurrentKafkaListenerContainerFactory<String, ClaimEvent>
factory(
    ConsumerFactory<String, ClaimEvent> cf) {
```

```

    ConcurrentKafkaListenerContainerFactory<String, ClaimEvent>
factory =
    new ConcurrentKafkaListenerContainerFactory<>();
factory.setConsumerFactory(cf);
factory.setConcurrency(3); // One thread per partition

// Retry with backoff
DefaultErrorHandler errorHandler = new DefaultErrorHandler(
    new DeadLetterPublishingRecoverer(kafkaTemplate),
    new ExponentialBackOffWithMaxRetries(3)
);
factory.setCommonErrorHandler(errorHandler);
return factory;
}

@KafkaListener(topics = "claims-events", groupId = "claims-group",
    concurrency = "3")
public void consumeClaim(ClaimEvent event,
    @Header(KafkaHeaders.RECEIVED_PARTITION)
    int partition) {
    log.info("Processing claim from partition {}", partition);
    claimsService.process(event);
}

```

10. Schema Evolution / Backward Compatibility

Issue: Producer sends new schema; old consumers break.

Resolution:

```

// Use Avro + Schema Registry
@Bean
public ProducerFactory<String, GenericRecord> producerFactory() {
    Map<String, Object> props = new HashMap<>();
    props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,

```



```
        KafkaAvroSerializer.class);
    props.put("schema.registry.url", "http://schema-registry:8081");
    props.put("auto.register.schemas", false); // Enforce
    compatibility check
    return new DefaultKafkaProducerFactory<>(props);
}
```

- Always use **BACKWARD** compatibility in Schema Registry
- Add new fields with defaults; never remove required fields

PART 2: 50 KAFKA INTERVIEW Q&A

SECTION A — Core Concepts (Q1–Q10)

Q1. What is Kafka's core architecture?

Kafka is a distributed commit log. Producers publish messages to **Topics**, which are split into **Partitions**. Each partition is an ordered, immutable sequence of records. Brokers store partitions; a **Zookeeper/KRaft** cluster manages metadata and leader election. Consumer Groups read from partitions in parallel.

Q2. What is a partition and why does it matter?

A partition is the unit of parallelism in Kafka. Each partition is consumed by exactly one consumer in a group. More partitions = higher throughput. Partition count also determines max consumer parallelism.

Rule: **consumer threads ≤ partition count**

Q3. Explain ISR (In-Sync Replicas).

ISR is the set of replicas fully caught up with the partition leader. A replica is removed from ISR if it falls behind by more than **replica.lag.time.max.ms**. With **acks=all**, the producer waits for all ISR replicas to acknowledge before the write is considered committed. This prevents data loss on broker failure.

Q4. What is the difference between acks=0, acks=1, and acks=all?

Setting	Guarantee	Risk
<code>acks=0</code>	Fire and forget	Message loss possible
<code>acks=1</code>	Leader acknowledges	Loss if leader crashes before replication
<code>acks=all</code>	All ISR replicas	Slowest but safest

Use `acks=all` + `min.insync.replicas=2` for production.

Q5. What is the role of the Consumer Group?

A consumer group allows horizontal scaling of consumption. Each partition is assigned to exactly one consumer in the group. Multiple groups can independently read the same topic. This enables pub-sub (multiple groups) and queue semantics (single group).

Q6. What is Log Compaction?

Log compaction retains the latest value for each key indefinitely, deleting older records with the same key. Useful for maintaining state (e.g., user profiles, claim status). Set with `cleanup.policy=compact`.

Q7. What is the difference between `at-most-once`, `at-least-once`, and `exactly-once`?

- **At-most-once:** Commit offset before processing → possible loss
- **At-least-once:** Commit after processing → possible duplicates
- **Exactly-once:** Use Kafka Transactions + idempotent producer + `isolation.level=read_committed`

Q8. How does Kafka ensure ordering?

Kafka guarantees ordering **within a partition**. Messages with the same key always go to the same partition (via hash partitioning), preserving per-key order. For global ordering, use a single partition (sacrifices parallelism).

Q9. What is `__consumer_offsets`?

An internal Kafka topic where consumer group offsets are stored. It replaced Zookeeper-based offset storage. Each entry maps `(group, topic, partition) → offset`.

Q10. What is KRaft mode?

KRaft (Kafka Raft) is Kafka's built-in metadata management, replacing Zookeeper. Introduced in Kafka 2.8, GA in 3.3+. A set of Kafka brokers act as a Raft-based quorum controller, eliminating Zookeeper dependency.

SECTION B — Producer Deep Dive (Q11–Q20)

Q11. What happens internally when `producer.send()` is called?

1. Message is serialized
 2. Partitioner determines target partition
 3. Message added to in-memory `RecordAccumulator` batch
 4. Sender thread picks up batch when `linger.ms` expires or batch is full (`batch.size`)
 5. Request sent to broker leader for that partition
 6. Broker writes to log, replicates to ISR, sends `Ack`
-

Q12. What is `linger.ms` and when should you increase it?

`linger.ms` is how long the producer waits to fill a batch before sending. Default is 0 (send immediately). Increasing it (e.g., 5–20ms) increases latency slightly but dramatically improves throughput by enabling larger batches. Use when throughput > latency priority.

Q13. What is idempotent producer and how does it work?

```
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
```

Each producer gets a PID (Producer ID) and a sequence number per partition. Broker deduplicates if it receives the same (PID, partition, sequence). Prevents duplicates from producer retries. Automatically sets `acks=all` and `max.in.flight.requests.per.connection=5`.

Q14. How do Kafka transactions work?

```
props.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "txn-claims-1");
producer.initTransactions();
producer.beginTransaction();
producer.send(record1);
producer.send(record2);
producer.commitTransaction(); // or abortTransaction()
```

Consumers set `isolation.level=read_committed` to only see committed messages. Used in Kafka Streams and exactly-once pipelines.

Q15. What is the impact of `max.in.flight.requests.per.connection`?

Controls how many unacknowledged requests a producer can have simultaneously. Set to `1` for strict ordering with retries (older approach). With idempotent producer enabled, Kafka 1.0+ supports up to `5` without reordering.

Q16. How do you implement a custom partitioner?

```
public class ClaimTypePartitioner implements Partitioner {
    @Override
    public int partition(String topic, Object key, byte[] keyBytes,
                        Object value, byte[] valueBytes, Cluster
cluster) {
        int numPartitions = cluster.partitionCountForTopic(topic);
        String claimType = key.toString();
        if ("DENTAL".equals(claimType)) return 0;
        if ("MEDICAL".equals(claimType)) return 1;
        return Math.abs(claimType.hashCode()) % numPartitions;
    }
}

// Register: props.put(ProducerConfig.PARTITIONER_CLASS_CONFIG,
ClaimTypePartitioner.class)
```

Q17. What causes `BufferExhaustedException`?

The producer's send buffer (`buffer.memory`, default 32MB) is full because the application is producing faster than the broker can accept. Solutions:

- Increase `buffer.memory`
- Increase `batch.size`
- Tune `linger.ms`
- Back-pressure the upstream

Q18. What is `compression.type` and which should you use?

Type	CPU	Compression Ratio
<code>none</code>	None	None
<code>gzip</code>	High	Best
<code>snappy</code>	Low	Medium
<code>lz4</code>	Very Low	Medium
<code>zstd</code>	Medium	Best (modern)

Use `lz4` for low-latency, `zstd` for best overall in Kafka 2.1+.

Q19. How do you handle large messages in Kafka?

Kafka is not designed for large payloads (default max 1MB). Options:

- Store payload in S3/blob store; send only the reference key in Kafka
- Increase `message.max.bytes` (broker) and `max.request.size` (producer) — risky at scale
- Use Kafka's chunk-based approach with a stitching consumer

Q20. What is a sticky partitioner?

Introduced in Kafka 2.4. Without a key, round-robin sent one message per batch (wasteful). Sticky partitioner fills a batch to one partition before switching. Improves batching efficiency significantly for keyless messages.

SECTION C — Consumer Deep Dive (Q21–Q30)

Q21. What is the consumer poll loop and why is blocking it dangerous?

```
while (true) {  
    ConsumerRecords<String, String> records =  
    consumer.poll(Duration.ofMillis(100));  
    // NEVER block here for long  
    processRecords(records); // Must complete before next poll  
}
```

If `processRecords` takes longer than `max.poll.interval.ms`, the broker assumes the consumer is dead and triggers a rebalance. Move slow processing to async threads and commit offsets carefully.

Q22. What is the difference between `commitSync` and `commitAsync`?

	<code>commitSync</code>	<code>commitAsync</code>
Blocks	Yes	No
Retries on failure	Yes	No (callback only)
Use case	End of batch, shutdown	High throughput loops

Best practice: use `commitAsync` in the loop, `commitSync` on shutdown/rebalance.

Q23. How do you handle rebalance gracefully?

```
consumer.subscribe(topics, new ConsumerRebalanceListener() {  
    @Override  
    public void onPartitionsRevoked(Collection<TopicPartition>  
    partitions) {
```

```
        // Commit offsets for revoked partitions before rebalance
        consumer.commitSync(currentOffsets);
    }

    @Override
    public void onPartitionsAssigned(Collection<TopicPartition>
partitions) {
        // Reset state, seek to specific offset if needed
    }
});
```

Q24. What is static group membership?

```
props.put(ConsumerConfig.GROUP_INSTANCE_ID_CONFIG, "claims-consumer-
pod-1");
```

Normally, a consumer restart causes a rebalance. With a static `group.instance.id`, the consumer rejoins with the same identity and reclaims its partitions without triggering a full rebalance. Ideal for Kubernetes pods with stable names.

Q25. How do you seek to a specific offset?

```
consumer.subscribe(Collections.singletonList("claims-events"));
consumer.poll(Duration.ofMillis(0)); // Trigger partition assignment

TopicPartition tp = new TopicPartition("claims-events", 0);
consumer.seek(tp, 5000L); // Resume from offset 5000

// Or seek to beginning/end
consumer.seekToBeginning(Collections.singletonList(tp));
consumer.seekToEnd(Collections.singletonList(tp));
```

Q26. How do you implement a pause/resume pattern for backpressure?

```

Set<TopicPartition> pausedPartitions = new HashSet<>();

while (true) {
    ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(100));
    for (TopicPartition partition : records.partitions()) {
        if (isDownstream Slow()) {
            consumer.pause(Collections.singleton(partition));
            pausedPartitions.add(partition);
        }
    }
    // Resume when downstream recovers
    if (isDownstreamHealthy()) {
        consumer.resume(pausedPartitions);
        pausedPartitions.clear();
    }
}
}

```

Q27. What is `fetch.min.bytes` and `fetch.max.wait.ms`?

- `fetch.min.bytes` (default 1): Broker waits until it has at least this many bytes before responding. Increase for throughput.
- `fetch.max.wait.ms` (default 500ms): Max wait time even if min.bytes not met. Controls latency floor.

Tune together: high `fetch.min.bytes` + high `fetch.max.wait.ms` = batch-oriented, high throughput.

Q28. How do you reset a consumer group offset?

```

# Reset to earliest offset
kafka-consumer-groups.sh --bootstrap-server broker:9092 \
    --group claims-group \
    --topic claims-events \

```



```
--reset-offsets --to-earliest --execute

# Reset to specific datetime (for incident replay)
kafka-consumer-groups.sh --bootstrap-server broker:9092 \
  --group claims-group \
  --topic claims-events \
  --reset-offsets --to-datetime 2025-01-15T10:00:00.000 --execute
```

Q29. What is Consumer Group Coordinator?

One of the Kafka brokers acts as the Group Coordinator for each consumer group (based on hash of group ID). It manages group membership, triggers rebalances, and stores offsets in `__consumer_offsets`.

Q30. How do you monitor consumer lag in production?

```
# CLI
kafka-consumer-groups.sh --bootstrap-server broker:9092 \
  --group claims-group --describe

# Programmatic (Spring Actuator + Micrometer)
management.metrics.tags.application=claims-service
# Expose: kafka.consumer.fetch-manager-metrics.records-lag-max
```

Also use Confluent Control Center, Datadog, or Prometheus `kafka_consumergroup_lag` metric.

SECTION D — Broker & Cluster (Q31–Q40)

Q31. How does Kafka leader election work?

Each partition has one leader broker. When a broker fails, the Controller (a special broker) detects it via Zookeeper/KRaft heartbeat timeout and promotes the most up-to-date ISR replica as the new leader. Producers/consumers then discover the new leader via metadata refresh.

Q32. What is `min.insync.replicas` and why is it critical?

```
min.insync.replicas=2
```

With `acks=all`, the write succeeds only if at least `min.insync.replicas` replicas acknowledge it. If only 1 replica is in sync and `min.insync.replicas=2`, the producer gets `NotEnoughReplicasException`. Prevents silent data loss. Standard production setting: `replication.factor=3`, `min.insync.replicas=2`.

Q33. What is unclean leader election?

```
unclean.leader.election.enable=false # Recommended
```

If `true`, a broker not in ISR can be elected leader — risks data loss (lagging replica becomes authoritative). If `false`, Kafka waits for an ISR member to become available — preserves durability but may cause availability gap.

Q34. How does Kafka handle back-pressure?

Kafka does not have native back-pressure. Producers get blocked when `buffer.memory` is full (`max.block.ms`). Solutions:

- Rate-limit producers at application level
 - Consumer pause/resume pattern
 - Increase partition count to spread load
 - Use reactive consumers (Spring WebFlux + Reactor Kafka)
-

Q35. What is the Kafka Controller?

One broker is elected as the Controller (via Zookeeper or KRaft). It manages:

- Partition leader election
- Broker joins/leaves
- Topic creation/deletion
- ISR changes

In KRaft mode, a quorum of 3+ brokers act as Raft voters for controller duties.

Q36. What causes `UnderReplicatedPartitions` alert and how do you fix it?

Causes:

- Broker crash or slow broker
- Network issues between brokers
- Disk I/O overload

Fix:

```
# Check under-replicated partitions
kafka-topics.sh --bootstrap-server broker:9092 --describe \
  --under-replicated-partitions

# Reassign partitions away from slow broker
kafka-reassign-partitions.sh ...
```

Q37. How do you increase partitions for an existing topic?

```
kafka-topics.sh --bootstrap-server broker:9092 \
  --alter --topic claims-events \
  --partitions 12
```

Warning: Increasing partitions changes the hash mapping for keyed messages → breaks ordering for existing keys. Never decrease partitions.

Q38. What is Rack Awareness in Kafka?

Configuring `broker.rack=us-east-1a` on each broker. Kafka then places partition replicas in different racks/AZs to survive zone failures. Combined with `replica.selector.class=RackAwareReplicaSelector` for consumers to prefer local-rack reads.

Q39. What is tiered storage in Kafka?

Introduced in Kafka 3.6. Offloads older log segments to object storage (S3/GCS/Azure Blob) while keeping recent data on broker disk. Enables infinite retention without massive broker disk capacity.

Q40. How do you tune Kafka for low latency vs high throughput?

Goal	Producer Config	Consumer Config
Low Latency	<code>linger.ms=0</code> , <code>batch.size=1</code> , <code>compression=none</code>	<code>fetch.min.bytes=1</code> , <code>fetch.max.wait.ms=0</code>
High Throughput	<code>linger.ms=20</code> , <code>batch.size=65536</code> , <code>compression=lz4</code>	<code>fetch.min.bytes=50000</code> , <code>fetch.max.wait.ms=500</code>

SECTION E — Spring Kafka & Advanced (Q41–Q50)

Q41. How does Spring Kafka's @KafkaListener work internally?

Spring creates a `KafkaMessageListenerContainer` per listener. It runs a dedicated thread per partition (if `concurrency` > 1) that polls and dispatches to your method. The container manages the consumer lifecycle, offset commits, and error handling via `CommonErrorHandler`.

Q42. How do you implement retry with dead-letter topic in Spring Kafka?

```
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<Object,
Object> template) {
    // Publish to <original-topic>.DLT after 3 retries
    DeadLetterPublishingRecoverer recoverer =
        new DeadLetterPublishingRecoverer(template,
            (record, ex) -> new TopicPartition(record.topic() +
                ".DLT", -1));

    ExponentialBackOffWithMaxRetries backOff =
        new ExponentialBackOffWithMaxRetries(3);
    backOff.setInitialInterval(1000L);
}
```

```

    backOff.setMultiplier(2.0);
    backOff.setMaxInterval(10000L);

    return new DefaultErrorHandler(recoverer, backOff);
}

```

Q43. How do you implement a Kafka Saga pattern for distributed transactions?

```

Claim Submitted → claims-events
  → ClaimsService publishes ClaimCreated
  → EligibilityService consumes, publishes EligibilityChecked
  → PaymentService consumes, publishes PaymentProcessed /
PaymentFailed
  → On failure: CompensationService consumes, rolls back

```

Each step is a Kafka message. Compensation events implement rollback. No 2PC needed. Use a Saga Orchestrator (separate service) or Choreography (event-driven).

Q44. What is Kafka Streams and how does it differ from a regular consumer?

	Regular Consumer	Kafka Streams
Stateless ops	Yes	Yes
Stateful ops (aggregation, join)	Manual	Built-in with RocksDB state store
Windowing	Manual	Built-in
Fault tolerance	Manual	Automatic (changelog topics)
Topology	Linear	DAG

```

StreamsBuilder builder = new StreamsBuilder();
KStream<String, ClaimEvent> stream = builder.stream("claims-
events");

```

```
stream.filter((k, v) -> v.getStatus().equals("PENDING"))
    .to("pending-claims");
```

Q45. How do you test Kafka consumers in Spring Boot?

```
@SpringBootTest
@EmbeddedKafka(partitions = 1, topics = {"claims-events"})
class ClaimsConsumerTest {

    @Autowired
    private KafkaTemplate<String, String> template;

    @Test
    void testConsumeClaimEvent() throws Exception {
        template.send("claims-events", "claim-001",
            "{\\"claimId\\":\\"C001\\",\\"status\\":\\"PENDING\\"}");

        // Wait for async processing
        await().atMost(5, SECONDS)
            .until(() ->
                claimsRepository.findById("C001").isPresent());
    }
}
```

Q46. How do you handle schema evolution without downtime?

1. Use Avro/Protobuf with Schema Registry
2. Always add new fields with **default** values
3. Use **BACKWARD** compatibility: new consumer reads old messages
4. Deploy consumers first, then producers (rolling deploy)
5. Never rename or remove fields without a major version bump

Q47. What is Reactor Kafka and when should you use it?

Reactor Kafka wraps the standard Kafka client with Project Reactor's reactive API. Use when your entire stack is reactive (Spring WebFlux):

```
ReceiverOptions<String, ClaimEvent> receiverOptions =
    ReceiverOptions.<String, ClaimEvent>create(props)
        .subscription(Collections.singleton("claims-events"));

KafkaReceiver.create(receiverOptions)
    .receive()
    .flatMap(record -> claimsService.processReactive(record.value())
        .doOnSuccess(r -> record.receiverOffset().acknowledge()))
    .subscribe();
```

Q48. How do you implement multi-tenancy in Kafka?

Options:

- **Topic-per-tenant:** `claims-tenantA`, `claims-tenantB` — simple, isolated, hard to scale
- **Partition-per-tenant:** Key = tenantId → same partition always; limited to partition count
- **Header-based routing:** Single topic, filter by Kafka header `X-Tenant-Id` in consumer

```
// Producer: tag with tenant
ProducerRecord<String, ClaimEvent> record = new ProducerRecord<>
    ("claims", key, event);
record.headers().add("tenantId", tenantId.getBytes());
```

Q49. How do you secure Kafka in production?

Encryption in transit

security.protocol=SSL

ssl.truststore.location=/certs/kafka.truststore.jks

ssl.keystore.location=/certs/kafka.keystore.jks

```
# Authentication
sasl.mechanism=SCRAM-SHA-512
security.protocol=SASL_SSL

# Authorization (ACLs)
authorizer.class.name=kafka.security.authorizer.AclAuthorizer
```

```
# Grant claims-service read access
kafka-acls.sh --bootstrap-server broker:9092 \
  --add --allow-principal User:claims-service \
  --operation Read --topic claims-events
```

Q50. What Kafka metrics should you monitor in production?

Metric	Alert Threshold	What It Means
<code>kafka.consumer.records-lag-max</code>	> 10,000	Consumer falling behind
<code>UnderReplicatedPartitions</code>	> 0	Replica health issue
<code>ActiveControllerCount</code>	!= 1	Controller problem
<code>OfflinePartitionsCount</code>	> 0	Critical — data unavailable
<code>BytesInPerSec</code>	> 80% capacity	Broker overloaded
<code>ProducerRequestLatency99thPercentile</code>	> 500ms	Slow broker or network
<code>FetchConsumerTotalTimeMs</code>	> 1000ms	Slow consumer fetches
<code>ISRShrinkRate</code>	> 0 sustained	Replica lag / broker issues

Expose via JMX + Prometheus JMX Exporter → Grafana dashboard.

Quick Reference — Critical Production Config

```
// PRODUCER — Safe, High Throughput
props.put(ProducerConfig.ACKS_CONFIG, "all");
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
```



```
props.put(ProducerConfig.RETRIES_CONFIG, Integer.MAX_VALUE);
props.put(ProducerConfig.LINGER_MS_CONFIG, 5);
props.put(ProducerConfig.BATCH_SIZE_CONFIG, 65536);
props.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "lz4");

// CONSUMER – Safe, Reliable
props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);
props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, 500);
props.put(ConsumerConfig.MAX_POLL_INTERVAL_MS_CONFIG, 300000);
props.put(ConsumerConfig.SESSION_TIMEOUT_MS_CONFIG, 30000);
props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
props.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG, "read_committed");

// BROKER – Durability
// replication.factor=3
// min.insync.replicas=2
// unclean.leader.election.enable=false
// log.retention.hours=168
```

6/2/2026, 5:49:07 PM